Leena M

Texas, US | +1 (972) 776-4448 | leenakrishna94@gmail.com

Professional Summary

Senior Consultant with 8+ years of experience translating enterprise workflows into production-oriented software, data, and integration solutions. Brings hands-on capability across **Python application development**, **Generative AI integration**, and **RAG and agentic workflows**, including prompt design, structured outputs, API contracts, retrieval, validation, testing, and secure data handling. Connects structured and unstructured data to usable services, investigates failure modes, and communicates trade-offs across application, data, DevOps, QA, cybersecurity, architecture, and business teams.

Technical Skills

AI Application Engineering: Python, Generative AI, LLM integration, Amazon Bedrock, boto3, prompt engineering, structured outputs, evaluation

RAG and Intelligent Search: RAG, embeddings, semantic search, vector retrieval, PostgreSQL, pgvector, intelligent search pipelines, citation grounding

Agents and APIs: AI agents, Strands Agents, agentic workflows, FastAPI, Pydantic, REST APIs, tool calling, schema validation

Enterprise Data and Documents: Structured and unstructured data integration, Amazon Textract, document extraction, data normalization, metadata and lineage

Quality, Security, and Governance: Automated testing, evaluation datasets, output validation, traceability, responsible AI, data protection, access controls, secure data handling

Cloud and Operations: AWS, Docker, PostgreSQL, observability, latency and token optimization, cost monitoring, reliability, deployment and support handoff

Professional Experience

Senior Consultant | Elevance Health

February 2026 - Present

- Translate stakeholder workflows and exception paths into Python service requirements, acceptance criteria, and production delivery slices for enterprise AI-assisted applications.
- Design bounded Generative AI capabilities on **Amazon Bedrock** with explicit user goals, source boundaries, fallback behavior, and human review points.
- Create reusable prompt templates with versioned instructions, structured outputs, and evaluation cases so changes can be compared before release.
- Build retrieval flows using **embeddings**, metadata filters, and **PostgreSQL/pgvector** to ground answers in approved business content.
- Add **citation checks**, source freshness checks, and abstention behavior when retrieved evidence is incomplete or contradictory.
- Implement **FastAPI** endpoints with **Pydantic** request and response models, making validation and error contracts visible at the service boundary.
- Use **boto3** clients and least-privilege service permissions to connect application flows to AWS-managed AI and document-processing capabilities.
- Prototype **AI agents** and bounded tool workflows with explicit tool schemas, authorization checks, timeout behavior, and human escalation.
- Integrate structured records and unstructured documents, preserving identifiers and provenance as information moves through extraction and retrieval stages.
- Apply **Amazon Textract** for document extraction where layout-aware text and table capture are more reliable than plain text ingestion.
- Design validation paths for malformed model responses, unavailable dependencies, empty retrieval results, and conflicting source records.
- Create automated tests for prompt regressions, schema adherence, retrieval grounding, tool authorization, sensitive-data exposure, and API failure handling.
- Compare latency, token usage, retrieval depth, and response quality to identify optimization opportunities; an illustrative review target is a 20% reduction in avoidable model calls.
- Instrument request, retrieval, tool, and model stages with trace identifiers so reviewers can follow a response from input through evidence and final output.
- Define output-quality review criteria covering factual support, completeness, refusal behavior, tone, and safe handling of ambiguous requests.
- Use **Docker** to package repeatable local and test environments, reducing setup differences across application, QA, and integration work.

- Document data contracts, prompt versions, model assumptions, deployment dependencies, rollback steps, and operational ownership for support teams.
- Coordinate with architecture, cybersecurity, QA, DevOps, data, and business partners to resolve dependencies before production release.
- Investigate failed responses by separating retrieval, prompt, model, authorization, data-quality, and downstream-service causes rather than treating every issue as model drift.
- Deliver review-ready implementation notes and test evidence for an illustrative 30-case evaluation set spanning normal, edge, adversarial, and abstention scenarios.

Senior Consultant | Optum

October 2024 - February 2026

- Develop Python components that connect business workflows, enterprise data, and AI-enabled assistance while keeping deterministic rules in control of high-risk decisions.
- Design **RAG** pipelines that split, enrich, embed, retrieve, and rank content before passing a bounded context to an LLM.
- Use **PostgreSQL** and **pgvector** schemas for document chunks, embedding metadata, source identifiers, and retrieval diagnostics.
- Build **FastAPI** services that expose stable REST contracts for ingestion, search, generation, evaluation, and reviewer feedback.
- Apply **Pydantic** models for request validation, typed structured outputs, field-level constraints, and predictable client errors.
- Implement prompt templates for classification, extraction, summarization, question answering, and tool selection with reusable variables and version history.
- Run prompt and retrieval evaluations against representative questions, measuring grounding, completeness, citation coverage, abstention, and invalid-output rates.
- Add response validation and repair logic for missing fields, unsupported claims, malformed JSON, and citations that do not match retrieved sources.
- Build agent-style workflows with explicit step boundaries, tool allowlists, retry limits, and escalation paths for unresolved tasks.
- Integrate document intake and **Amazon Textract** extraction with downstream normalization, confidence checks, and exception queues.
- Use **Amazon Bedrock** model calls through **boto3**, centralizing configuration for model selection, temperature, token limits, and request timeouts.
- Connect unstructured content to structured business records while preserving record keys, document versions, and processing status for traceability.
- Create automated unit, integration, regression, and negative tests covering prompt injection, unauthorized tools, sensitive data, empty context, and dependency failures.
- Add exponential backoff, idempotency keys, and dead-letter handling to ingestion and model-dependent workflows to avoid duplicate or silently lost work.
- Profile retrieval depth, token counts, service latency, and cache opportunities; an illustrative test goal is a 15% improvement in median response latency.
- Define structured logs and trace identifiers for request, retrieval, model, and tool stages while excluding sensitive payloads from routine telemetry.
- Review AI controls with cybersecurity and governance partners, including authorization, retention, data minimization, audit evidence, and responsible-use boundaries.
- Package services with **Docker** and document environment variables, secrets handling, health checks, and deployment prerequisites for DevOps handoff.
- Maintain architecture notes, evaluation results, prompt changes, known limitations, and support procedures so decisions remain explainable after delivery.
- Collaborate with application, data, QA, and business teams to move an illustrative evaluation from 25 representative cases to a repeatable release gate.

Consultant | LatentView

June 2022 - August 2024

- Analyze business processes and data flows to identify where Python automation, search, or AI-assisted review can reduce manual investigation without removing accountable review.
- Develop reusable Python utilities for ingestion, normalization, validation, matching, comparison, and repeatable data preparation.
- Build REST integrations with explicit request contracts, response handling, timeout behavior, and actionable error messages.
- Support retrieval experiments using document metadata, text chunking, embeddings, and similarity search to improve access to unstructured information.
- Prepare prompt templates for extraction, classification, summarization, and question answering, keeping instructions separate from dynamic context.
- Evaluate generated outputs for factual support, completeness, consistency, safe refusal, and usefulness using representative examples and reviewer feedback.
- Implement schema checks and comparison rules that catch missing fields, unexpected types, duplicate records, and inconsistent source values.

- Create test cases for normal paths, empty inputs, malformed responses, unavailable services, and authorization failures across integrated workflows.
- Document source lineage, processing assumptions, data-quality exceptions, and reviewer decisions so outputs can be reproduced and challenged.
- Coordinate with data and application contributors to align API payloads, source identifiers, transformation logic, and downstream expectations.
- Investigate processing discrepancies by tracing inputs, transformations, service calls, and output records rather than relying on surface-level symptoms.
- Use containerized development patterns and repeatable configuration to reduce environment drift between implementation and QA activities.
- Summarize technical trade-offs, delivery risks, and unresolved dependencies for stakeholders making prioritization and rollout decisions.
- Measure illustrative workflow scope through a 10-source comparison set and document the assumptions behind any quality or time-saving estimate.
- Prepare deployment notes, troubleshooting guidance, rollback considerations, and support handoff materials for operational teams.

Consultant | Reylon Softech

June 2018 - June 2022

- Gather requirements from business users and convert process needs into implementation tasks, validation rules, and testable acceptance criteria.
- Build and maintain **Python** application integrations and API consumers with explicit input contracts, error handling, and recovery paths.
- Develop SQL transformations for normalization, deduplication, comparison, reconciliation, and repeatable reporting workflows.
- Create data-quality checks for missing values, invalid formats, unexpected duplicates, and mismatched source records.
- Implement REST request and response handling with timeouts, status checks, logging, and clear escalation behavior for failed dependencies.
- Write unit and regression tests that trace delivered behavior to requirements, acceptance criteria, and known failure scenarios.
- Support document and text-processing workflows by preserving source identifiers, processing status, and reviewer-visible exception details.
- Document technical decisions, operating procedures, dependency assumptions, and support guidance for delivery and stakeholder teams.
- Communicate implementation trade-offs, risks, and unresolved issues clearly while escalating decisions requiring accountable business or security owners.
- Track an illustrative 12-case regression set across data, API, and validation paths to make repeatable checks part of the delivery handoff.

Education

Master's in Cybersecurity | Saint Louis University

May 2026